University Singidunum

Faculty of Technical Sciences

Design, Fabrication, and Control of a Low-Cost Quadruped Robot with Integrated Computer Vision

Bachelor Thesis

Mentor: Aleksandar Jevremović

Student: Roman Sannikov, 2023230586

Belgrade, 2026

Contents

1 Introduction **2**

1.1 Literature review 3

2 Methodology **6**

2.1 Methods and technologies used 6

2.1.1 Project requirements 6

2.1.2 Technology selection 7

2.1.3 Implementation 10

2.1.4 Object Detection Model 17

3 Results **26**

4 Conclusion **27**

5 Future work **28**

Reference list **29**

Chapter 1

Introduction

Over the past four decades, there has been a major leap in robotics, especially in the field of quadruped robots. Many important papers developing the field have been published, leading to significant advancements. The development progressed from early one-legged hopping machines to four-legged trotting robots able to traverse difficult terrain and obstacles, and later to autonomous navigation in unstructured environments. Legged locomotion has shown itself to be one of the most versatile forms of mobility. However, most of this research was conducted in laboratories, requiring substantial resources for prototyping and pushing the boundaries of the field. This creates a gap between laboratory research and consumer-level accessibility.

The objective of this thesis is to address that gap by designing, constructing, and controlling a quadruped robot as an integrated project. The work covers the full development pipeline: mechanical design using modern CAD tools and 3D printing, selection and characterization of actuators, implementation of statically stable gaits based on established stability criteria, development of a control system, integration of computer vision, and preparation for physical validation. By following this end-to-end process, the thesis aims to demonstrate how classical locomotion and computer vision principles can be realized on a budget-friendly platform.

A deliberate choice was made to use low-cost, commercially available components and rapid-prototyping techniques wherever possible. This decision was not driven by cost reduction alone; it also helps ensure that the resulting platform is reproducible. In this paper, I review research and practical achievements made throughout the years and combine them in one project

at a lower cost. The idea is to establish a base for low-cost quadruped robots and make them more affordable for day-to-day use in different fields of everyday life.

1.1 Literature review

There are several "classical" papers that can be used as a basis for this project. They cover topics ranging from gaits and control to computer vision and successful real-world systems.

The most important capability of a quadruped robot is locomotion. The mathematical analysis in the 1968 *Mathematical Biosciences* article "On the stability properties of quadruped creeping gaits" shows that there is "a unique optimum gait that maximizes static stability" [3]. The work explains that static stability requires the projection of the center of gravity onto the ground to remain within the support polygon. This polygon is formed by connecting all feet that are in contact with the ground, where each foot represents one contact point.

However, stability can also be achieved by dynamic means. The concept of dynamic stability is discussed in the 1984 article "Experiments in Balance with a 3D One-Legged Hopping Machine", which describes control algorithms for forward velocity, body attitude, and hopping height [4]. Unlike static locomotion, dynamic locomotion allows the center of gravity to move outside the support region for short periods. Balance is then maintained through continuous motion and active control rather than through a permanently stable body position. Moving the projection of the center of gravity outside the support region causes the robot to start falling. During this falling motion, the robot places its feet to catch itself, and repeating this sequence creates a dynamic gait. In other words, dynamic stability is continuous falling and catching.

In recent years, computing power has become more accessible, while the development of neural networks has enabled the creation of increasingly realistic simulated environments. Digital versions of robots can be placed in these environments and given a task, such as moving from point A to point B, without receiving an explicitly programmed sequence of joint movements. Instead, the robot learns a movement policy by interacting with the simulated environment through Deep Reinforcement Learning (DRL). This approach was used for the ANYmal robot in the 2019 article "Learning Agile and Dynamic Motor Skills for Legged Robots" [5]. The researchers incorporated the real-world behavior of the robot's motors into the simulation. This made the simulation more realistic and helped the learned policy account for motor imperfec-

tions and other differences between the simulated and physical systems.

While the theoretical foundations establish how quadruped locomotion works, several landmark systems demonstrate what is achievable with mature engineering. Boston Dynamics' BigDog demonstrated load-carrying dynamic locomotion on rough terrain and was one of the first systems to show that quadrupeds could operate in real-world conditions outside the laboratory [1]. It directly continued ideas from the 1984 hopping-machine experiments and made it evident that dynamic stability is the key to traversing challenging terrain. Similarly, MIT's Cheetah 3 showed that model-based control could achieve robust blind locomotion over obstacles [2]. These systems represented state-of-the-art quadruped capabilities at the time. The projects were sponsored by DARPA and MIT, respectively, and did not focus on cost reduction, but rather on pushing the boundaries of what quadruped robots could do.

Another key component of the project is computer vision. Generally, robots can be controlled by a human operator, which remains a very practical approach. However, computer vision is a significant step towards autonomy and high-level task execution. Vision is critical for navigation, obstacle avoidance, and interaction with the environment. For this project, I decided to use YOLO, a real-time object detection model based on the You Only Look Once architecture [6]. The original model divides an image into a grid, and each grid cell is responsible for predicting bounding boxes, confidence scores, and class probabilities for an object whose center falls into that cell. As described in the paper: "[The] network architecture is inspired by the GoogLeNet model for image classification... [It] has 24 convolutional layers followed by 2 fully connected layers. Instead of the inception modules used by GoogLeNet,... [it simply uses] 1×1 reduction layers followed by 3×3 convolutional layers..." [6]. The original YOLO uses a Sum-Squared Error (SSE) loss function between predictions and ground truth. While the architecture has evolved significantly, some later YOLO versions adopted CSPDarknet, a high-performance convolutional neural network backbone, together with more advanced loss functions. Other versions use different backbones, feature-aggregation methods, and detection heads. Nevertheless, single-pass detection remains a central principle.

Additionally, it is worth mentioning "Object Detection with Discriminatively Trained Part-Based Models" [7]. This classical paper describes how objects can be detected from their individual parts. The concept is relevant to this project because the robot's low camera position may allow it to see only the lower part of a person or another object. Training the model to

recognize partially visible objects helps the robot detect objects from its unusual viewpoint.

While the model architecture and the approach to training it are important, the datasets used for training enable the model to reach its full potential. In this project, I utilize two of the most significant benchmarks in the history of object detection: Common Objects in Context (COCO) [9] and Stanford Dogs [10]. COCO established the standard for detecting objects in "natural" settings rather than perfectly centered studio shots. It contains more than $330,000$ images, of which over $200,000$ are labelled, with approximately 1.5 million object instances across 80 object categories, including people, dogs, and cats. The Stanford Dogs dataset contains $20,580$ annotated images of dogs belonging to 120 breeds. Each image is annotated with a bounding box and a class label.

Chapter 2

Methodology

2.1 Methods and technologies used

Developing a functional quadruped robot on a limited budget requires navigating a wide landscape of hardware components, software frameworks, and design methodologies. While the robotics field offers a rich variety of actuators, computing platforms, sensors, and control strategies, the project constraints significantly narrow the range of viable options. Cost, reproducibility, and the mechanical demands of legged locomotion impose trade-offs at every stage of the design process, forcing a careful balance between performance, affordability, and ease of integration. The following subsections outline the requirements that shaped this design space and present the technologies and models selected to address them.

2.1.1 Project requirements

There are several requirements and limitations in this project, as it seeks to demonstrate a way to develop a low-cost quadruped robot. Three main groups of limitations define the requirements. The first, and most significant, is cost. It influences the second group, which is hardware. The hardware, limited by cost, in turn constrains the software and algorithms. Together, they create a tight framework for the project, which is beneficial because it encourages creativity and finding new solutions.

My requirements for the project are the following:

- The robot should be able to walk on flat surfaces in all directions: forward, backward, sideways, and turning in place. Flat surfaces were selected because rough terrain is too challenging for the hardware used in this project.

- The robot should have 12 degrees of freedom (DOF), where each joint that can move independently contributes one DOF, to allow for a variety of gaits and maneuvers. Each leg should have 3 DOF: hip abduction and adduction (side-to-side movement), hip flexion and extension (forward-backward movement), and knee flexion and extension.

- It should have computer vision capabilities to detect objects. For the sake of demonstration, I chose people and dogs as the target classes because the robot is expected to encounter and interact with them.

- The robot should be easy to assemble and reproduce by others. The components should be available, affordable, and designed with swappability in mind to allow future upgrades and repairs.

On the limitation side, the total cost of the project should not exceed 300 EUR. This significantly limits what the project can achieve.

The hardware and software choices are affected by this limitation, and the technology selection is discussed in the next section.

2.1.2 Technology selection

Everything starts with the model. Nowadays, 3D printers are quite accessible, making them a great choice for this project because they allow custom parts to be produced according to the design requirements while remaining reproducible by others. At my disposal were the Bambu Lab A1 mini, X1 Carbon, and H2D printers. I tried them all and settled on the H2D because it has two nozzles, allowing PLA and PETG to be used in the same print. PLA is a suitable general-purpose material for the robot itself, while PETG is used for support interfaces. It is possible to use only PLA, and I tested that approach, but PETG makes support removal easier because it does not bond strongly to PLA. I designed the robot in Onshape, a free cloud-based CAD web application with sufficient functionality for this project. For the slicer, I used Bambu Studio, which works well with the printers and provides the required functionality. A slicer

converts 3D models into G-code, specifying toolpaths, layer heights, support placement, and other print parameters.

I did not create the initial model myself, as doing so would require much more time. Instead, I used a suitable model by Tom Knox, published under a Creative Commons 4.0 license, and modified it to meet the requirements [11]. The requirements not met by the initial model were 12 DOF and ease of assembly. What the original model did provide was the base design I wanted, three DOF in each front leg, and a place for the camera module.

What made the original 3D model hard to assemble was the fact that it consisted of 149 unique parts connected by different types of screws. The most problematic part was the leg. All four legs are the same, just mirrored, which made redesigning much easier. To reduce assembly complexity, I redesigned the legs and body to be print-in-place, meaning ready-to-use parts without the need for additional assembly. I also reduced the number of unique screws in the project to one type, M3, in two sizes.

For the actuators, I chose WS-MG996R digital servos with metal gears, which are widely used in hobby robotics. Three servos per leg provide 12 DOF in total. These servos are affordable and provide a specified stall torque of $9.4\,\mathrm{kgf \cdot cm}$ at 4.8 V and $11\,\mathrm{kgf \cdot cm}$ at 6 V. The latter is approximately $1.08\,\mathrm{N \cdot m}$. Their specified speed is 0.17 seconds per 60° at 4.8 V and 0.14 seconds per 60° at 6 V, and their specified operating voltage range is 4.8–7.2 V. However, these low-cost actuators have limited precision and torque, and they do not expose position feedback to the controller. Considering these limitations, they are the minimum suitable option for this project.

For the main controller, I chose a Raspberry Pi 4 Model B with 4 GB of RAM. It is powerful enough to control the servos and run a small object detection model. It uses a 64-bit Broadcom BCM2711 quad-core Cortex-A72 processor running at up to 1.8 GHz and a VideoCore VI GPU with support for OpenGL ES 3.1 and Vulkan 1.0. It also provides dual-band Wi-Fi, Bluetooth 5.0 BLE, Gigabit Ethernet, two USB 3.0 ports, two USB 2.0 ports, a 40-pin GPIO header, a two-lane MIPI CSI camera port, a two-lane MIPI DSI display port, USB-C power input, and 4 GB LPDDR4-3200 SDRAM. A big advantage of this computer is that it runs Raspberry Pi OS, a Linux-based operating system that allows fast prototyping and the use of existing programs and development tools [12].

The Raspberry Pi provides enough general-purpose pins for 12 control signals, but only a small number support hardware PWM. Software-generated PWM under Linux can also have timing jitter. I therefore used a PCA9685, a 16-channel, 12-bit PWM driver that generates stable servo signals in hardware. It communicates over I2C using two signal lines: SDA on GPIO 2 and SCL on GPIO 3. It is also connected to 3.3 V for logic power and to a common ground [13].

The specified stall current is 2.5 A per servo at 6 V. Using this value, a theoretical simultaneous stall of all 12 servos would require 30 A, although this is not the normal operating current. To power the robot, I decided to use a battery pack made from four 2500 mAh 18650 lithium-ion batteries. Connected in a 2S2P configuration (two series pairs in parallel), they provide a nominal voltage of 7.4 V and a total nominal capacity of 5000 mAh. Since the Raspberry Pi requires a 5 V supply, I use buck converters to provide 7 V for the servos and 5 V for the Raspberry Pi.

For computer vision, I use a 5-megapixel Raspberry Pi Camera Module v1.3, which features an OmniVision OV5647 color CMOS sensor. The camera has a 3.6 mm focal length lens with a $54^\circ \times 41^\circ$ angle of view, providing a horizontal field of view roughly equal to the distance from the subject at typical operating ranges. The camera uses a CSI ribbon cable to connect to the dedicated MIPI CSI port on the Raspberry Pi, offering lower communication overhead than a typical USB camera.

On the software side, I use Python as the main programming language. It is a great choice for rapid prototyping and has a large ecosystem of libraries for robotics, computer vision, and machine learning. For controlling the servos, I use the Adafruit PCA9685 library, which provides a simple interface for generating PWM control signals. PWM control does not regulate the servo's supply voltage. Instead, the pulse width encodes the requested position. Typical control pulses range from approximately 1 to 2 milliseconds and are repeated at a fixed rate. The exact pulse-to-angle range depends on servo calibration. For object detection, I use the Ultralytics YOLO library, which offers a streamlined interface for YOLO training and inference and integrates well with the Raspberry Pi platform.

2.1.3 Implementation

The implementation started by downloading the original model and printing its most important part: the leg. The original leg model consisted of seven parts and took around three hours to print. Afterward, I assembled the leg, attached the hip to a servo, and connected the servo to the Raspberry Pi with jumper wires for an initial test. Printing and testing the leg was a crucial step that helped me verify that the model could move. Having the physical model also helped me understand what needed to be changed.

Later, I started redesigning the leg to be print-in-place. First of all, it is important to understand the principles of such a design. The most important idea is that unsupported overhangs should be minimized, meaning that the parts should be supported by the print itself where possible. To achieve this, I used 45° as a practical overhang limit, beyond which support material was generally required. Another thing to keep in mind is the clearance between the moving parts. Through experimentation, I found that 0.65 mm provides the best balance between smooth motion and minimal play, so this value was used for most clearances in the model. The layer height for every print was 0.2 mm, and support material was enabled for overhanging parts such as the curves of the legs and the body "ribs".

The final leg consists of six elements: motor extension, extension link, shoulder disk, femur, secondary link, and tibia, printed as one module. After removing it from the printer bed and detaching the supports, it is movable and ready to be installed. To get to the final design, I went through around twenty iterations of the leg and a couple of iterations of the body. Several design challenges emerged during these iterations, and to achieve the requirements while using the same design, I had to make exceptions to the basic principles. For example, a support blocker is used to prevent support from being generated inside the motor extension, creating a bridging surface that spans a gap without support.

Another example showing the challenge of designing print-in-place models is the knee. The knee joint employs a cylindrical pivot pin that is integrated into the tibia (lower leg), with the femur (upper leg) captured between the two halves of the tibia joint, forming a sandwich-like structure. This arrangement creates overhangs on both the top and bottom surfaces of the femur where it interfaces with the tibia. Support material is therefore required in two places: between the upper tibia surface and the femur, and between the femur and the lower tibia surface. The

overhanging surface is large and could droop and fuse to the other part, after which it could not be recovered. Support was therefore necessary, so unfortunately, each knee contains a small amount of PETG support material. PLA can also be used as an alternative support material, but it must be broken free by moving the knee joint after printing.

Beyond the main structural parts, the model contains eight servo holders that are printed separately. Each holder weighs 5 grams and does not require support material. The holders attach to the servos with screws, while the connection to the body uses integrated pins that slot into corresponding holes, eliminating the need for additional fasteners. In total, printing the complete model requires approximately 20.5 hours and 490 grams of filament. The body takes 7 hours and 154 grams, each of the four legs requires 3 hours and 74 grams, and the eight servo holders together take 1.5 hours and 40 grams.



Figure 2.1: Partially completed printing process of an experimental leg on a Bambu Lab A1 3D printer using only PLA filament. The tibia is on the left, support for the femur and secondary link is in the middle, and the motor extension, extension link, and shoulder disk are at the top. The image demonstrates how the print-in-place leg is designed internally.

When the whole model was stable and ready, I started assembling it all together. With the experience I gained, I estimate that it would take around one hour to assemble everything. Connecting the PCA9685 to the Raspberry Pi was a straightforward process. The control-pin connection layout is the following: VCC to 3.3 V, GND to the common ground, SDA to GPIO 2, and SCL to GPIO 3. The PCA9685 logic supply does not power the servos; instead, the board has separate V+ and GND terminals for the servo power supply. These are connected to

an external power supply. While testing, I used a laboratory power supply set to 6 V with a 6 A current limit, which was enough for testing.

In order to enable the I2C communication interface on the Raspberry Pi, I enabled it through `sudo raspi-config` under Interface Options, I2C. After that, I installed the I2C utilities by running `sudo apt install i2c-tools`. The connection can be verified by running `i2cdetect -y 1`, which should show the PCA9685 at its default 7-bit address, `0x40`. In the I2C protocol, each device requires a unique 7-bit address so that the controller can identify it. The PCA9685 uses a partially configurable address scheme: the most significant bit is fixed, while the lower six bits can be modified using the external address pins A^0 through A^5. With all six pins low, the binary address is 1000000, corresponding to `0x40`. This design allows up to 62 PCA9685 boards to coexist on the same bus. For example, setting A^0 high while keeping the other pins low changes the address to `0x41`. For this project, I use the default address, `0x40`.

The other pins form 16 channels for PWM output, which are connected to the servos. Each channel consists of three pins: V+ for power, GND, and OUT for the control signal. I grouped them by leg in the following order: hip extension, knee flexion, and hip abduction, so it is easier to understand which channel controls each joint. The PCA9685 board provides sockets for the servos, so they can be connected without soldering. However, the board exposes the I2C pins, while the Raspberry Pi exposes its GPIO header, so I soldered wires to connect them.

Each servo has a 180° range of motion, so before attaching the servos to the robot, I set the hip extension to 90°, knee flexion to 60°, and hip abduction to 95° as their home positions. These values are defined in the code, and all movements are relative to them. Then, the servos are attached as follows. The hip-abduction servo is fixed vertically in its slot in the body, and a dual-arm horn is attached to its output shaft. After that, the hip-extension and knee-flexion servos are placed together and secured with servo holders on the sides. Another dual-arm horn is attached to the knee-flexion servo, while a round disc horn is fixed to the rear side of the femur. Finally, the horns are connected to the hip and motor extension, respectively. The same procedure is repeated for all four legs. After that, the electronics are installed inside the body, and the Raspberry Pi camera module is mounted in the front camera slot. With the mechanical assembly complete, the control software can command movement.

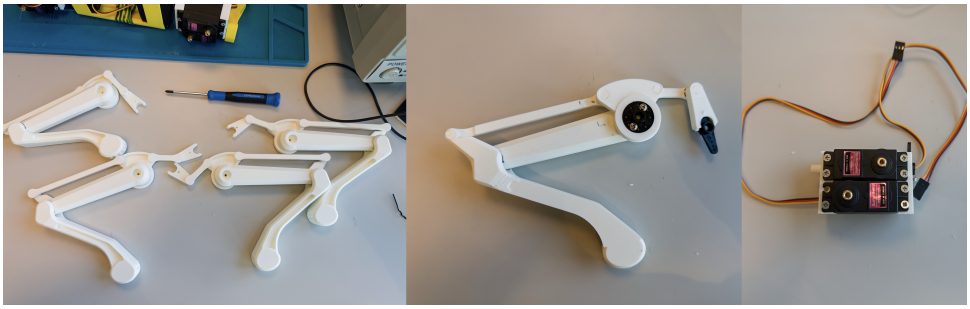


Figure 2.2: On the left are the front sides of the legs. In the middle is the back side of a leg with connected horns. On the right are the hip-abduction and knee-flexion servos attached together with the servo holders.

A step consists of two phases: swing and stance. During the swing phase, the foot leaves the ground and moves to the next contact position. During the stance phase, the foot remains in contact with the ground and supports the robot. For forward movement, it moves backward relative to the chassis, generating the ground reaction force that pushes the robot's body forward. A gait defines how the steps of all four legs are timed and coordinated.

In this implementation, the swing and stance phases are represented using trajectories. During the swing trajectory, the foot moves linearly in the horizontal direction while its vertical position follows a sine-shaped arch:

$$x(t) = x_{\text{start}} + (x_{\text{end}} - x_{\text{start}})t, \qquad y(t) = y_{\text{ground}} + L\sin(\pi t), \tag{2.1}$$

where t ranges from 0 to 1 and L is the lift height. This arch is smooth and resembles a parabola, but the implementation uses a sine function rather than a mathematical parabola. For the same lift height and step duration, the sine profile has a lower vertical speed at lift-off and touchdown than a parabolic profile, making these transitions less abrupt. During the stance trajectory, the horizontal position changes linearly in the opposite direction while y remains at y_{ground}.

To translate desired foot positions into servo angles, the system uses a lookup-table approach rather than analytical inverse kinematics. This method was chosen because the mechanical coupling in the leg design, where hip movement affects the effective knee geometry, makes a simple analytical model impractical. The lookup table contains pre-calibrated mappings between joint angles and foot positions, created by setting different servo angles and measuring the resulting foot positions. With the hip mounting point defined as the coordinate origin, the

foot position is expressed in millimeters relative to this reference. The calibration data consists of 81 measurements spanning the feasible range of hip and knee angles. Each entry records the hip angle, knee angle, and the corresponding foot coordinates, capturing the reachable workspace of the leg. When the control system requests a target foot position (x_t, y_t), it locates the nearest calibration points in the two-dimensional position space and interpolates between them using inverse distance weighting. For each calibration point i with foot position (x_i, y_i), the Euclidean distance to the target is computed as

$$d_i = \sqrt{(x_t - x_i)^2 + (y_t - y_i)^2}. \tag{2.2}$$

The four nearest points are selected, and a weight is assigned to each according to the inverse square of its distance:

$$w_i = \frac{1}{d_i^2}. \tag{2.3}$$

These weights are normalized so that they sum to unity,

$$\hat{w}_i = \frac{w_i}{\sum_{j=1}^{4} w_j}, \tag{2.4}$$

and the interpolated joint angles are obtained as the weighted average of the corresponding calibration angles:

$$\theta_{\text{hip}} = \sum_{i=1}^{4} \hat{w}_i \, \theta_{\text{hip},i}, \qquad \theta_{\text{knee}} = \sum_{i=1}^{4} \hat{w}_i \, \theta_{\text{knee},i}. \tag{2.5}$$

In practice, the implementation includes several practical adjustments: if the target lies within 0.1 mm of a calibration point, the stored angles are returned directly without interpolation; distances below 0.001 mm are clamped to prevent division by zero; and the final interpolated angles are rounded to the nearest integer because the current software represents commands in whole degrees. This is a software choice, not a limitation of the PCA9685 or servo. The method assigns greater influence to closer calibration points, producing interpolated angle estimates for positions that were not explicitly measured. The approach is particularly suitable for this mechanism because the mechanical coupling and manufacturing tolerances make an accurate closed-form analytical model difficult to derive. While inverse kinematics is the conventional approach in robotics literature, empirical mapping provides a practical alternative when mechanical complexity or manufacturing tolerances make analytical models unreliable.

The abduction joints, which control side-to-side leg movement, are not included in the lookup table because their motion is kinematically decoupled from the primary leg swing mechanism. These joints operate independently and are controlled through direct angle commands. Each servo is individually calibrated to account for mechanical variations and assembly differences between left and right legs. A direction multiplier ensures that positive commands produce consistent physical movements across all legs, while center offsets compensate for variations in the home position. Three hip-and-knee gait functions are implemented. In the crawl function, the legs swing in the order left front, right rear, right front, and left rear. Because the function moves one leg at a time, three feet remain on the ground, although the actual static stability still depends on the center-of-gravity position. In the trot gait, the left-front and right-rear legs form one diagonal pair, while the right-front and left-rear legs form the other. One pair follows the swing trajectory while the other follows the stance trajectory, and the pairs then exchange roles. In the turning gait, the left and right legs use opposite horizontal directions to generate rotational torque.

The sideways gait uses the hip-abduction joints to move the legs laterally. First, the robot tilts its body away from the direction of movement, shifting its weight onto the supporting legs. The legs on the other side are then moved outward one at a time. After that, the robot tilts its body in the opposite direction, allowing the remaining legs to move inward one at a time. Repeating this sequence moves the robot sideways while three legs support the body during each step.

The software architecture is organized into four functional layers. The lowest layer abstracts the hardware interface, managing communication with the PWM driver and providing a unified interface for setting servo angles. Above this sits the kinematics layer, which implements the lookup table and interpolation logic to translate desired foot positions into joint angles. The gait generation layer defines the periodic leg movement patterns, swing and stance trajectories, and orchestrates their execution across multiple legs according to the selected gait pattern. As discussed in Section 1.1, static stability requires the projection of the center of gravity to remain within the support polygon formed by the grounded feet. Moving one leg at a time keeps three feet on the ground, so the crawl gait is designed to meet this condition. However, the condition still depends on the actual center-of-gravity position and must be verified experimentally. The trot gait moves diagonal pairs simultaneously and trades static stability for speed. The highest layer comprises the application interfaces that allow human operators to command the robot.

The primary control interface is a web server, which runs on the Raspberry Pi and exposes HTTP-based remote-control endpoints. HTTP is a request-response application protocol operating over TCP, in which the client (the operator's browser) initiates a connection and sends a request, and the server returns a response. The server listens on a dedicated port and handles two types of requests. A GET request to the root path returns the HTML control page, which contains a JavaScript component that captures keyboard events. When the operator presses a key, the script issues an asynchronous POST request to `/moveForward`, `/moveBackwards`, `/walkSidewaysLeft`, or `/walkSidewaysRight`. The server parses the request path to determine the desired action and, because HTTP itself is stateless, maintains an internal state machine to track whether the robot is idle, standing, or executing a gait. Upon receiving a movement command, the server transitions to the moving state and spawns a background execution thread so that the gait can run without blocking the communication interface. A timeout mechanism automatically returns the robot to a resting position if no further commands are received for three seconds, preventing the robot from remaining in an active stance indefinitely. This design prioritizes responsiveness and simplicity, making the robot accessible to operators without requiring specialized control hardware or software installation. For development and debugging, separate command-line utilities allow individual servos to be positioned manually. These tools were essential during calibration and mechanical assembly because they enable direct control of any joint without starting the full gait system. They were also used to set the servos to the angles required for collecting lookup-table measurements.

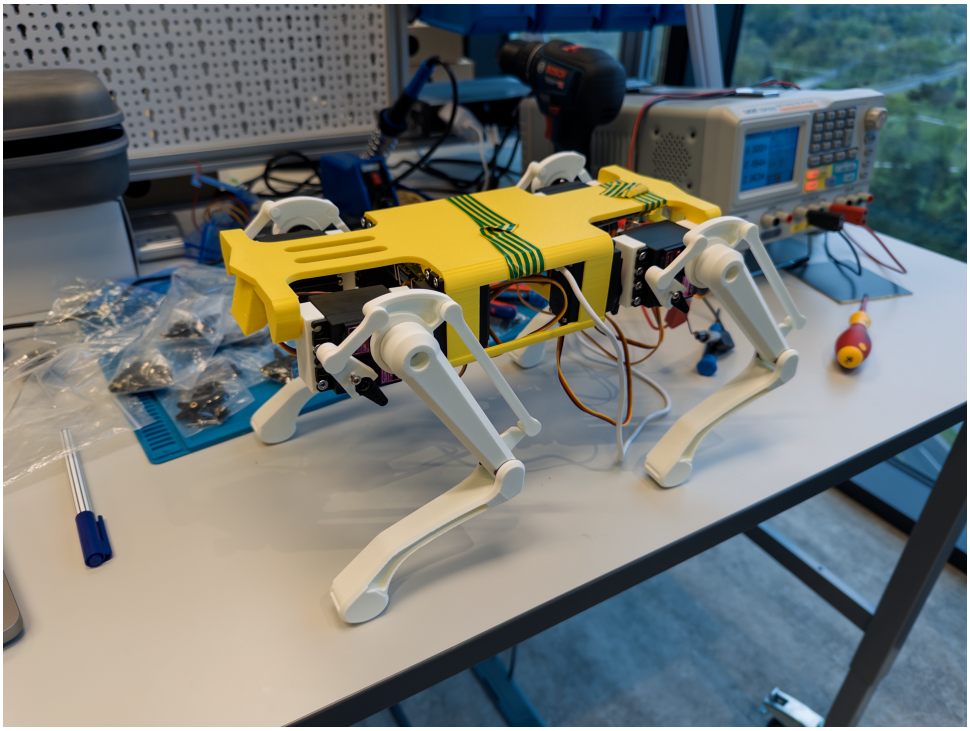


Figure 2.3: Semi-assembled standing quadruped robot without a camera module, powered by a lab power supply, with a power plug behind it. The wires are not yet organized.

2.1.4 Object Detection Model

Before discussing the YOLO architecture, it is important to understand the underlying technology that makes it possible: the Convolutional Neural Network (CNN). A CNN is a class of deep neural networks specifically designed for processing grid-structured data such as images. Unlike traditional fully connected networks, where every neuron is connected to every neuron in the adjacent layer, a CNN employs convolutional layers that apply learnable filters (kernels) across local regions of the input. Each filter detects specific visual features of a given image, such as edges, textures, shapes, or increasingly abstract patterns, by computing dot products between the filter weights and the local pixel values. The output of this operation is a feature map that preserves the spatial relationships within the image. CNNs can also incorporate pooling layers, which progressively reduce the spatial dimensions of the feature maps, thereby decreasing computational complexity and providing a degree of translation tolerance. This hierarchical feature extraction, combined with weight sharing across spatial locations, makes CNNs exceptionally effective for computer vision tasks while requiring far fewer parameters than fully connected networks.

The original YOLO introduced an innovative approach to object recognition [6]. Before it, systems such as the Deformable Parts Model (DPM) [7] and Region-based Convolutional Neural Network (R-CNN) [8] used classifiers in multi-stage detection pipelines. Classifiers for each object were used to evaluate it at various locations and scales in a test image, either in a sliding-window manner or on proposed bounding boxes. A post-processing stage would then refine the bounding boxes and remove duplicate detections. Such cumbersome pipelines were slow and difficult to optimize because of the heterogeneous nature of their components. The new approach introduced by YOLO uses the whole image to train the model to predict objects, converting object detection into a single regression problem. This method uses a single convolutional network to predict multiple bounding boxes and their class probabilities simultaneously, making it swift by simplifying the pipeline. Unlike the aforementioned mechanisms, YOLO can reason globally by examining the whole image, which gives it the advantage of learning generalized representations of objects.

The following description applies to the original YOLO model. The model starts by dividing the input image into an $S \times S$ grid. Each grid cell is responsible for predicting bounding boxes, their confidence scores, and class probabilities. Grid cells are not limited to one bounding box; instead, they can predict B boxes but only one set of C conditional class probabilities per cell. In the original paper, the authors use $B = 2$ and $C = 20$ on a 7×7 grid. Each bounding box is made up of five numbers: the x and y positions, representing the location of the center relative to the grid cell; the width and height relative to the entire image; and the confidence score, representing the predicted Intersection over Union (IoU) between the bounding box and the ground truth, multiplied by the probability that an object is present. Formally,

$$\text{confidence} = \Pr(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}. \tag{2.6}$$

If no object exists in the cell, the confidence score should ideally be zero. The class probabilities are conditioned on the cell containing an object, denoted $\Pr(\text{class}_i \mid \text{object})$. At test time, the class-specific confidence for each box is obtained by multiplying the conditional class probabilities by the box confidence:

$$\Pr(\text{class}_i \mid \text{object}) \times \Pr(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}} = \Pr(\text{class}_i) \times \text{IoU}_{\text{pred}}^{\text{truth}}. \tag{2.7}$$

This product encodes both the probability that the box contains an object of class i and how well the box fits the object.

The original network architecture borrows heavily from GoogLeNet, also known as Inception-v1 [6]. It is made up of 24 convolutional layers followed by two fully connected layers. The convolutional layers handle feature extraction, starting with simple edges and textures and building up to more complex patterns, while the fully connected layers map these features to the final output tensor of size $S \times S \times (B \times 5 + C)$.

For training, the authors chose Sum-Squared Error (SSE) as the loss function. This single loss covers three things at once: the accuracy of bounding box coordinates, the confidence score, and the class prediction. A challenge here was that most grid cells in any image do not contain an object, which can bias the model toward always predicting "no object". To prevent this, the loss function applies different weights to different parts of the prediction. Coordinate errors are multiplied by $\lambda_{\text{coord}} = 5$ to increase their importance, while confidence errors for empty boxes are scaled down by $\lambda_{\text{noobj}} = 0.5$. This ensures that the network pays more attention to getting the boxes right when objects are actually present.

Because every cell predicts multiple bounding boxes, the network naturally generates many overlapping detections around the same object. To deal with this, YOLO applies non-maximum suppression (NMS) as a post-processing step. NMS works by selecting the box with the highest confidence and removing lower-confidence boxes that overlap with it significantly, meaning boxes whose IoU exceeds a chosen threshold. The end result is usually a single high-confidence bounding box per object rather than multiple overlapping predictions.

Since its original release, YOLO has evolved through several major architectural shifts. Early iterations moved from pure regression to anchor-based detection, introduced batch normalization and multi-scale training, and adopted deeper residual backbones with feature pyramid-style predictions at multiple resolutions. The next wave of improvements focused on training efficiency and deployment accessibility: Cross-Stage Partial networks reduced computational cost, the ecosystem migrated from Darknet to native PyTorch, and automated augmentation and anchor calculation removed much of the manual tuning. More recently, the architecture has shifted toward anchor-free detection, decoupled classification and regression heads, and unified frameworks that integrate object detection with segmentation and pose estimation un-

der a single model family. For this project, I decided to use the Ultralytics YOLO library with the YOLOv8n model [14]. YOLOv8 uses an anchor-free detector and separate outputs for classification and bounding-box regression. I chose the nano variant to reduce memory use and inference time on the Raspberry Pi. Its actual on-device frame rate and detection performance must be measured experimentally.

As mentioned before in the project requirements, the selected target classes are dogs and humans. I chose two significant datasets in the history of object detection: COCO [9] and Stanford Dogs [10]. COCO contains more than $330,000$ images, of which over $200,000$ are labelled, and approximately 1.5 million object instances across 80 object categories. Stanford Dogs contains $20,580$ annotated images of dogs belonging to 120 breeds. Even though the datasets are suitable as they are, they must be adapted to the case I have, which is a close-to-ground quadruped robot. The average height of the camera in the standing position is approximately 17 cm, with an upward tilt of 15°.

To understand why adaptation is necessary, it helps to look at what the camera actually sees. The camera has a horizontal field of view of 54° and a vertical field of view of 41°. The optical axis is tilted 15° above horizontal, so the vertical field of view is split around this axis: the top edge of the frame is $15^\circ + 20.5^\circ = 35.5^\circ$ above horizontal, and the bottom edge is $15^\circ - 20.5^\circ = 5.5^\circ$ below horizontal.

Expressing all distances in centimeters, the height of the top edge at horizontal distance d is

$$y_{\text{top}} = 17 + d \cdot \tan(35.5^\circ) \approx 17 + 0.713d, \tag{2.8}$$

and the height of the bottom edge is

$$y_{\text{bottom}} = 17 - d \cdot \tan(5.5^\circ) \approx 17 - 0.096d. \tag{2.9}$$

The factor 0.713 comes from $\tan(35.5^\circ)$ and the factor 0.096 comes from $\tan(5.5^\circ)$. The total vertical span of the frame at distance d is the difference between these two values, which works out to approximately $0.809d$ cm. Anything outside the band $[y_{\text{bottom}}, y_{\text{top}}]$ is cut off.

The horizontal width of the view is determined by the 54° horizontal field of view. With half

of that angle being 27°, the total width is

$$w = 2d \cdot \tan(27^\circ) \approx 1.019d. \tag{2.10}$$

Figure 2.4 illustrates the geometry in a side view. The camera sits at 17 cm above the ground and looks upward at 15°. The vertical field of view of 41° is split into 20.5° above and below the optical axis, giving the top edge at 35.5° and the bottom edge at 5.5° below horizontal.

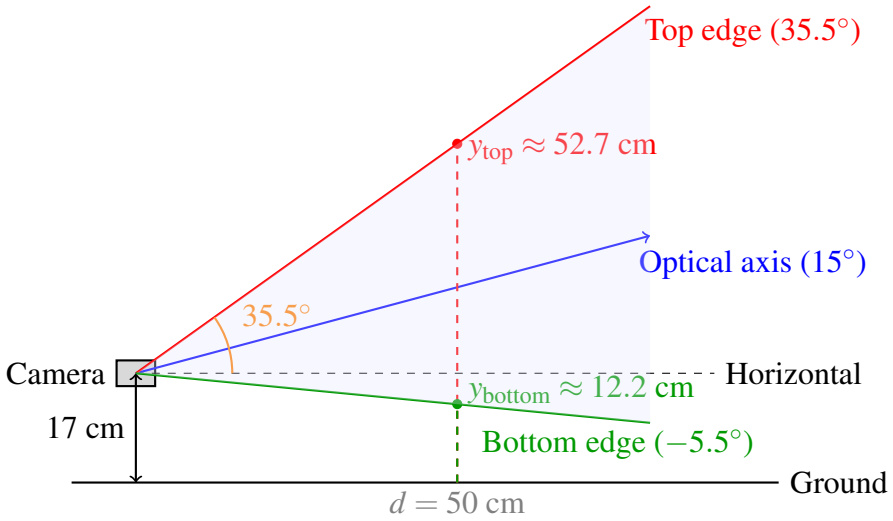


Figure 2.4: Side view of the camera geometry showing the vertical field of view at a distance of $d = 50$ cm. The shaded region represents the camera's view frustum.

Figure 2.5 shows the horizontal field of view from above. The camera sees a wedge-shaped region that widens with distance.

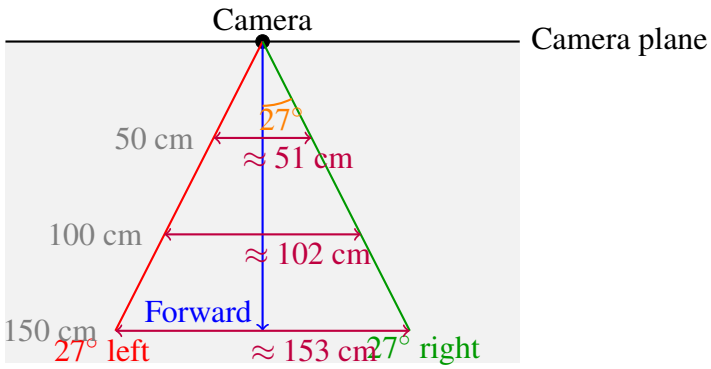


Figure 2.5: Top-down view of the horizontal field of view. The labels mark forward distances of 50 cm, 100 cm, and 150 cm, with the corresponding view widths shown in purple.

Table 2.1 shows the resulting numbers for a few typical distances. The "Visible height range" column gives the total vertical span of the frame ($y_{\text{top}} - y_{\text{bottom}}$). The "Human visible" and

"Dog visible" columns show what fraction of a standing human (170 cm tall) and a medium-sized dog (50 cm tall) actually falls inside that span. For example, at $d = 50$ cm the frame covers the band from 12.2 cm to 52.7 cm above the ground. A human standing there occupies $[0, 170]$ cm, so the overlap is $[12.2, 52.7]$ cm, giving a visible portion of 40.5 cm or about 24% of the total body. A dog of height 50 cm occupies $[0, 50]$ cm, so the overlap is $[12.2, 50]$ cm, giving 37.8 cm or about 76% of the body.

Distance	**Visible height range**	**Visible width**	**Human visible**	**Dog visible**
50	40.5	51.0	≈24%	≈76%
100	81.0	101.9	≈48%	≈85%
150	121.4	152.9	≈71%	≈95%

Table 2.1: Camera field of view at different distances (in cm), assuming an adult human height of 170 cm and a medium dog height of 50 cm.

The numbers make the problem clear. At the distances where the robot is likely to encounter humans and dogs, roughly 50–150 cm, the camera captures the lower portion of humans and most of a dog's body. At 50 cm, only about a quarter of a human is visible, mostly lower legs and feet. Dogs fare much better because of their smaller size, with three-quarters or more of the body typically in frame. As the distance increases, more of a human comes into view, but the perspective remains unusual: the camera looks up from below rather than straight on. This is a very different viewpoint from the standard dataset images, which are usually taken at eye level and show complete objects. Training on those images without adjustment would leave the model unprepared for the low-angle, from-below perspective that the robot actually experiences.

To address the issue, I created a balanced dataset by combining the original COCO and Stanford Dogs datasets. For every accepted annotation, the preparation code saves a full-object crop and a lower-body crop. The human crop contains a randomly selected lower 40–70% of the annotated height, while the dog crop contains the lower 60–80%. This does not reproduce a projective camera transformation; instead, it approximates the partial view produced by the robot's low camera position. Additional transformations vary brightness, contrast, blur, and scale, while YOLO's training augmentation also changes color, translation, scale, and image composition.

The current generated split contains $12,500$ training images and $2,500$ validation images, which is approximately an 83–17 split. The training set contains $5,000$ images with people,

5,000 with dogs, and 2,500 negative samples. The validation set contains 1,000 images with people, 1,000 with dogs, and 500 negative samples. Negative samples therefore form 20% of each current split.

The model is trained on a MacBook Pro with an Apple M2 Pro chip and a 16-core GPU. The training utilizes the PyTorch Metal Performance Shaders (MPS) backend, which is Apple's framework, built on the Metal graphics API, for accelerating machine learning workloads on the GPU [15, 16]. Architecturally, MPS abstracts the complexity of writing low-level Metal Shading Language by providing optimized operations and kernels. These perform parallel mathematical operations such as multidimensional matrix multiplication and convolution [15].

The execution of these operations is orchestrated by PyTorch's dynamic computational graph, which maps the sequence of mathematical operations and data dependencies required by the model. When the graph is dispatched to the `mps` device, PyTorch sends supported operations to the MPS backend for execution on the GPU [15]. Apple silicon uses a Unified Memory Architecture (UMA), in which the CPU and GPU access a shared pool of physical memory [17]. This can reduce the need to copy data between separate CPU and GPU memory pools. PyTorch's MPS backend maps supported graph operations to MPS Graph functions or optimized Metal kernels. The resulting GPU commands are encoded into Metal command buffers, which are scheduled for execution on the GPU. This tight integration of Apple's hardware with PyTorch allows the GPU to accelerate training compared with CPU-only execution.

An additional factor that contributes to the speed of training is the correct use of training parameters. The run that produced the validation image in Figure 2.6 can be identified by an exact file match with its saved artifact. Its configuration contains 30 epochs, a batch size of 8, image size 320, the MPS device, and no data-loading workers. The following command reconstructs its effective saved settings:

```
yolo detect train model=yolov8n.pt \
data=dataset.yaml \
imgsz=320 \
batch=8 \
epochs=30 \
device=mps \
```

```
workers=0 \
patience=100 \
optimizer=auto \
cache=False \
val=False \
mosaic=0
```

where:

`yolo detect train` The Ultralytics CLI command that selects the YOLO detection task and starts a training run.

`model=yolov8n.pt` The initial weights to load (YOLOv8 Nano). This uses a small, fast pre-trained model as a transfer-learning starting point.

`data=dataset.yaml` Path to the dataset configuration YAML describing training/validation image locations and class names.

`imgsz=320` Sets the input canvas to 320×320 pixels. Images keep their aspect ratio and are padded as needed. A smaller input reduces memory and computation but can reduce accuracy for small objects.

`batch=8` Number of images processed per optimization step. Lower batch sizes reduce memory use, but they do not always increase training speed and can reduce GPU throughput.

`epochs=30` Number of full passes over the training set.

`device=mps` Use the Apple Metal Performance Shaders (MPS) GPU backend on macOS instead of the CPU.

`workers=0` Load data in the main process rather than in worker subprocesses.

This configuration is well-optimized for training on consumer hardware while still achieving good accuracy. The training process typically takes around 2.5 hours to complete on this hardware. The recorded training curves show steady convergence: the bounding-box, classification, and distribution focal losses decrease consistently, indicating that the model learned from the data. The final validation produced very high aggregate metrics: precision $\approx 96.5\%$, recall

$\approx 94.5\%$, mAP@50 $\approx 98.7\%$, and mAP@50:95 $\approx 97.8\%$. At the same time, the validation classification loss is noticeably larger than the training classification loss (approximately 0.70 compared with 0.09), suggesting some degree of overfitting in the class predictions. The metrics describe performance on the prepared validation set; an independent set of images captured by the robot is still needed to measure generalization to the real camera viewpoint. These results are encouraging for a lightweight `yolov8n` model with a 320×320 input.



Figure 2.6: Final validation predictions from the 30-epoch run, with bounding boxes, classes, and confidence scores.

Chapter 3

Results

Chapter 4

Conclusion

Chapter 5

Future work

Bibliography

[1] Raibert, M., Blankespoor, K., Nelson, G., & Playter, R. (2008). BigDog, the rough-terrain quadruped robot. *Proceedings of the 17th IFAC World Congress*, 10822–10825.

[2] Katz, B., Di Carlo, J., & Kim, S. (2019). MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2245–2251.

[3] McGhee, R. B., & Frank, A. A. (1968). On the stability properties of quadruped creeping gaits. *Mathematical Biosciences*, 3(3–4), 331–351.

[4] Raibert, M. H. (1984). Experiments in balance with a 3D one-legged hopping machine. *The International Journal of Robotics Research*, 3(2), 75–92.

[5] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., & Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), eaau5872.

[6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[7] Felzenszwalb, P. F., Girshick, R. B., McAllester, D., & Ramanan, D. (2010). Object detection with discriminatively trained part-based models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(9), 1627–1645.

[8] Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *Proceedings of the IEEE Conference*

on Computer Vision and Pattern Recognition (CVPR), 580–587.

[9] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *Proceedings of the European Conference on Computer Vision (ECCV)*, 740–755.

[10] Khosla, A., Jayadevaprakash, N., Yao, B., & Fei-Fei, L. (2011). Novel dataset for fine-grained image categorization. *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*.

[11] Knox, T. (n.d.). Quadruped I. https://www.printables.com/model/1443108-quadruped-i

[12] Raspberry Pi Ltd. (n.d.). Raspberry Pi 4 Model B specifications. https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/

[13] NXP Semiconductors. (2015). PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller. https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf

[14] Ultralytics. (n.d.). YOLOv8 documentation. https://docs.ultralytics.com/models/yolov8/

[15] PyTorch Contributors. (2026). MPS backend. https://docs.pytorch.org/docs/stable/notes/mps.html

[16] Apple Inc. (n.d.). Metal framework. https://developer.apple.com/documentation/metal

[17] Apple Inc. (n.d.). Choosing a resource storage mode for Apple GPUs. https://developer.apple.com/documentation/metal/choosing-a-resource-storage-mode-for-apple-gpus